

JUnit Advanced Testing

Exercise 1: Parameterized Tests

Problem

Test an `isEven()` method using parameterized tests.

Setup

Create `EvenChecker.java` and `EvenCheckerTest.java`.

Java Implementation

```
// EvenChecker.java
public class EvenChecker{
    public boolean isEven(int n){ return n%2==0; }
}

// EvenCheckerTest.java
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;
import static org.junit.jupiter.api.Assertions.*;

public class EvenCheckerTest{
    EvenChecker e=new EvenChecker();

    @ParameterizedTest
    @ValueSource(ints={2,4,6,8,10})
    void testEven(int n){
        assertTrue(e.isEven(n));
    }
}
```

Output

Program executes successfully and all test cases pass.

Exercise 2: Test Suites and Categories

Problem

Group multiple test classes into a suite.

Setup

Create two test classes and a suite class.

Java Implementation

```
import org.junit.platform.suite.api.*;
```

```
@Suite
@SelectClasses({CalculatorTest.class, EvenCheckerTest.class})
public class AllTests{
}
```

Output

Program executes successfully and all test cases pass.

Exercise 3: Test Execution Order

Problem

Execute tests in a specified order.

Setup

Use `@TestMethodOrder` and `@Order`.

Java Implementation

```
import org.junit.jupiter.api.*;
```

```
@TestMethodOrder(MethodOrderer.OrderAnnotation.class)
public class OrderedTests{
```

```
    @Test
    @Order(1)
    void first(){ System.out.println("First"); }
```

```
    @Test
    @Order(2)
    void second(){ System.out.println("Second"); }
```

```
    @Test
    @Order(3)
    void third(){ System.out.println("Third"); }
}
```

Output

Program executes successfully and all test cases pass.

Exercise 4: Exception Testing

Problem

Verify that a method throws the expected exception.

Setup

Create ExceptionThrower.java and its test class.

Java Implementation

```
public class ExceptionThrower{  
    public void throwException(){  
        throw new ArithmeticException("Error");  
    }  
}
```

```
import org.junit.jupiter.api.Test;  
import static org.junit.jupiter.api.Assertions.*;
```

```
public class ExceptionThrowerTest{  
    @Test  
    void testException(){  
        ExceptionThrower e=new ExceptionThrower();  
        assertThrows(ArithmeticException.class,  
            ()->e.throwException());  
    }  
}
```

Output

Program executes successfully and all test cases pass.

Exercise 5: Timeout and Performance Testing

Problem

Ensure a method completes within a specified time.

Setup

Create PerformanceTester.java and its test class.

Java Implementation

```
public class PerformanceTester{  
    public void performTask() throws InterruptedException{  
        Thread.sleep(500);  
    }  
}
```

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class PerformanceTesterTest{
    @Test
    void testTimeout(){
        PerformanceTester p=new PerformanceTester();
        assertTimeout(java.time.Duration.ofSeconds(1),
            ()->p.performTask());
    }
}
```

Output

Program executes successfully and all test cases pass.